

Créer un bundle Git

Depuis PowerShell, on peut créer un fichier .bundle contenant l'historique Git commité d'une branche.

1



Ouvrir PowerShell dans le dossier du projet

2



Lancer la commande git bundle create

3



Vérifier que le fichier .bundle apparaît sur le Bureau

```
PS C:\Users\Etudiant\Desktop\video-lab> git bundle create  
"$HOME\Desktop\feature-video.bundle" feature/video-implementation  
  
PS C:\Users\Etudiant\Desktop\video-lab> |
```



Bundle créé : feature-video.bundle



Branche : feature/video-implementation



Historique Git commité uniquement



Important

Le bundle contient l'historique Git nécessaire pour cette branche.
Il ne contient aucun mot de passe GitHub ni token.

2

Téléverser le bundle dans Colab

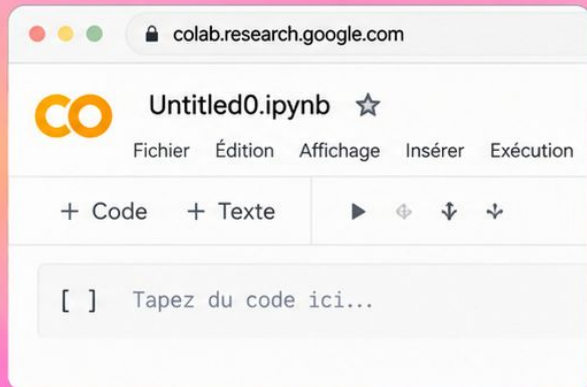
On envoie le fichier .bundle depuis son ordinateur vers l'environnement Colab.



1

Ouvrez un notebook Colab

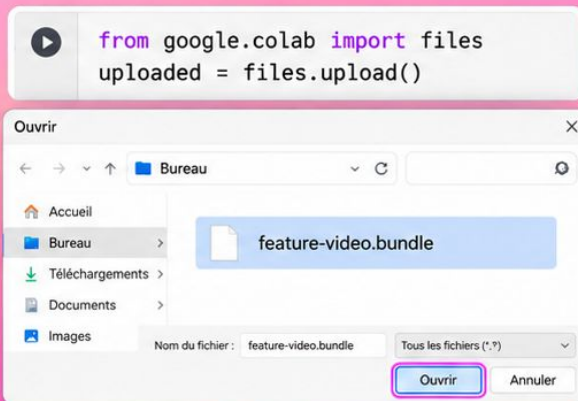
Allez sur colab.research.google.com et créez un nouveau notebook ou ouvrez-en un.



2

Téléversez le fichier

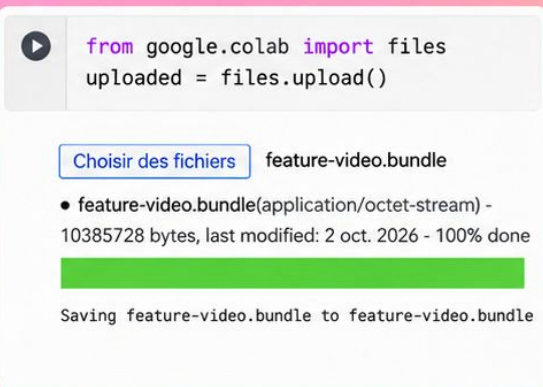
Exécutez cette cellule dans Colab pour ouvrir la fenêtre de sélection de fichier.



3

Vérifiez le téléversement

Une fois le fichier sélectionné, Colab téléverse le bundle et affiche le résultat.



Bon à savoir

Le fichier .bundle est simplement copié dans le dossier /content/ de votre environnement Colab. Il ne contient pas de mot de passe ou de token GitHub.



Résultat attendu

- ✓ Le fichier apparaît dans /content/
- ✓ Vous pouvez maintenant reconstruire le dépôt à partir de ce bundle.

3

Reconstruire le dépôt dans Colab

On recrée le dépôt à partir du fichier `.bundle` téléversé.



1

Préparer le code

Dans une cellule Colab, collez et exécutez ce code. Adaptez si besoin le nom du fichier et du dossier.



```
import subprocess
import shutil
from pathlib import Path

# Chemins (à adapter si besoin)
bundle = Path("/content/feature-video.bundle")
project = Path("/content/video-lab")
```

2

Reconstruire le dépôt

Ce code supprime l'ancien dossier (s'il existe), clone le dépôt depuis le bundle et se place sur la bonne branche.



```
# Supprimer l'ancien dossier s'il existe
if project.exists():
    shutil.rmtree(project)

# Cloner depuis le bundle
subprocess.run(["git", "clone", str(bundle),
               str(project)], check=True)

# Se placer sur la branche voulue
subprocess.run(
    ["git", "-C", str(project), "checkout",
     "feature/video-implementation"],
    check=True,
)
```

3

Exécuter et vérifier la sortie

La cellule affiche la progression du clonage et la confirmation du changement de branche.



```
subprocess.run(["git", "-C", str(project),
               "log", "-1", "--oneline"],
               check=True)
```

```
Cloning into '/content/video-lab'...
done.
Switched to a new branch 'feature/video-implementation'

8a1f96b Ajout de la génération vidéo (exemple)
```



Bon à savoir

Le bundle contient l'historique Git commité de la branche, ce qui permet de recréer un dépôt complet dans Colab. Il ne contient pas de mot de passe ni de token GitHub.



Résultat attendu



Le dépôt est recréé dans `/content/video-lab/`.



La branche `feature/video-implementation` est bien sélectionnée.

4

Vérifier le dépôt reconstruit

On vérifie que la bonne branche est active et que le dernier commit est bien présent.



1

Vérifier la branche actuelle

Cette commande affiche la branche dans laquelle se trouve le dépôt.



```
!git -C /content/video-lab \  
branch --show-current
```



Résultat (exemple)

```
feature/video-implementation
```

2

Voir le dernier commit

Cette commande affiche le dernier commit de la branche actuelle.



```
!git -C /content/video-lab \  
log -1 --oneline
```



Résultat (exemple)

```
8a1f96b Ajout de la génération  
vidéo (exemple)
```

3

Vérifier que le dossier existe

Vous pouvez aussi lister les fichiers pour confirmer que le projet est présent.



```
!ls /content/video-lab
```



Résultat (exemple)

```
notebooks/  
src/  
README.md  
requirements.txt  
data/  
...
```



Bon à savoir

Si la branche affichée et le dernier commit correspondent à ce que vous attendez, le dépôt a bien été reconstruit à partir du bundle et vous pouvez maintenant travailler sur vos fichiers dans Colab.



Résultat attendu

- ✓ La bonne branche est active.
- ✓ Le dernier commit est bien affiché.
- ✓ Le dossier du projet est présent dans /content/video-lab/.

5

Comprendre le résultat

On vérifie que le dépôt a bien été reconstruit et que tout est prêt pour travailler dans Colab.



1

La bonne branche est active

La commande affiche la branche actuelle du dépôt.



```
!git -C /content/video-lab \  
branch --show-current
```



Résultat (exemple)

```
feature/video-implementation
```



La branche affichée correspond à celle que vous attendez.

2

Le dernier commit est bien présent

La commande affiche le dernier commit de la branche actuelle.



```
!git -C /content/video-lab \  
log -1 --oneline
```



Résultat (exemple)

```
8a1f96b Ajout de la génération  
vidéo (exemple)
```



Le dernier commit correspond bien à l'historique attendu.

3

Le dépôt est bien présent

Vous pouvez lister le contenu du dossier pour vérifier que vos fichiers sont là.



```
!ls /content/video-lab
```



Résultat (exemple)

```
notebooks/  
src/  
README.md  
requirements.txt  
data/  
...
```

/content/video-lab

```
video-lab  
├── notebooks  
├── src  
├── README.md  
├── requirements.txt  
└── data  
...
```



Bon à savoir

Si toutes ces vérifications fonctionnent, cela signifie que le dépôt a bien été reconstruit dans Colab à partir du bundle. Vous pouvez maintenant ouvrir vos notebooks et travailler normalement.



Résultat attendu

- ✓ La bonne branche est active.
- ✓ Le dernier commit est bien affiché.
- ✓ Le dossier du projet est présent et contient vos fichiers.